

ML Specification Report — playground-series-s3e1

California Housing — Median House Value Regression • AIDE agent (20-step tree search, Claude Sonnet via shim)

All scores are taken from the run's `facts_aide.json`. Tree structure, per-step plans and per-node execution times come from `logs/2-adorable-spicy-kudu/journal.json`; hyperparameters from `best_solution.py` and the run's `config.yaml`; metric/target metadata from the competition `config.yaml`. Dataset shapes are counted directly from the run's own `input/` directory.

Contents	
California Housing — Median House Value Regression • AIDE agent (20-step tree search, Claude Sonnet via shim)	1
Overview	1
Data	2
Models & Architecture	3
Training procedures	4
Search trajectory	4
Optimization specification	5
Inference procedures	6
Evaluation & Benchmarking	7

Overview

The task is to predict a census block's median house value (`MedHouseVal`) on the California-Housing Playground Series episode (metric: RMSE, lower is better). AIDE attacks it as a **tree search over executable solutions**: it drafts a complete Python script, runs it, reads the printed metric or the traceback, and mutates the best node it has seen. Over **20 steps — 18 scored, 2 buggy** — it walked a Random-Forest baseline of **0.59196** down to a champion **local OOF RMSE of 0.54952**, found at **step 19, the very last step of the budget**. The champion is a repeated 2×5 -fold LightGBM + XGBoost + CatBoost blend over 40 features, whose distinguishing ingredient is a multi-scale out-of-fold KNN spatial target encoding that AIDE invented in the last three steps.

On the leaderboard this is AIDE's strongest episode of the batch relative to the other two lanes: **public 0.55863, private 0.55343, rank 218/690 (68.6th percentile)**, and the score table records `lb_winner = aide` — AIDE's private score beats both the my-agent lane (0.55987) and the NVIDIA reproduce lane (0.5555). The same table records `local_winner = nvidia`; the other lanes' local numbers are outside this report's source scope, so only the leaderboard comparison is reconstructed here.

Why it matters. House-value modelling sits under mortgage underwriting, property taxation and regional planning, and California Housing is the canonical benchmark for tabular geo-regression. For the agent comparison it matters for a second reason: it is the one episode in this batch where AIDE's search converged onto a genuinely comp-specific idea (local spatial target statistics) rather than on generic ensembling — and, as the Data section records, also the one episode whose input directory was not clean.

Data

Purpose of Data. Predict the median house value of a California census block from tabular census and geographic descriptors — a continuous-target **regression** episode scored by RMSE (minimize), with `MedHouseVal` as target and `id` as key. **Data Format** is clean, all-numeric **tabular CSV**. **Data Volume**, counted from the run's `input/` directory, is **37,137 training rows × 10 columns** and **24,759 test rows × 9 columns** — i.e. 8 raw predictive features (`MedInc`, `HouseAge`, `AveRooms`, `AveBedrms`, `Population`, `AveOccup`, `Latitude`, `Longitude`).

An input-purity caveat specific to this episode. The run's `input/` directory did not contain only the raw competition files. Alongside `train.csv` / `test.csv` / `sample_submission.csv` it held `train_processed.csv` (26 columns), `train_processed_v2.csv` (28 columns), `test_processed_v2.csv` (27 columns), `round1_best_lgb_params.json`, `round1_oof_test.npz` and `round2_oof_test.npz`. AIDE used them: from step 1 onward almost every node reads `train_processed_v2.csv` instead of `train.csv`, and the champion additionally loads its LightGBM hyperparameters straight out of `round1_best_lgb_params.json`. So on this episode AIDE did **not** build its feature set from raw data — 26 of the champion's 40 features and its entire LightGBM configuration were inherited from the working directory, and AIDE's own contribution is the 14 geospatial features it added on top. The other four episodes in this batch had raw-only or near-raw inputs; s3e1's numbers should be read with this caveat attached.

Data Quality is **not recorded**. AIDE ran no profiling step and produced no EDA artifact for this competition — the entire tree consists of model-fitting scripts. The only data-quality handling that exists on record is what AIDE wrote into code: at step 0 it replaced division-produced `inf` / `NaN` in its ratio features with column medians, and from step 1 onward it relied on the pre-processed CSVs being clean. Missing-value counts, duplicate counts and train↔test shift were never measured by this run.

Annotation Guidelines. The label is the continuous `MedHouseVal`; submissions are real-valued predictions scored by RMSE, with no class label, threshold or ranking involved.

Feature Set. The champion's matrix is 40 columns wide:

$$26 \text{ (inherited v2 features)} + 6 \text{ (haversine city distances)} + 8 \text{ (multi-scale KNN mean/std)} = 40$$

Group	Origin	Features
Raw + inherited engineering (26)	read from <code>train_processed_v2.csv</code>	8 raw columns plus <code>households</code> , <code>bedroom_ratio</code> , <code>rooms_per_person</code> , five <code>log_*</code> transforms, <code>dist_LA / dist_SF / dist_SanDiego / dist_Sacramento / dist_SanJose / dist_nearest_city</code> , <code>lat_long</code> , <code>geo_cluster</code> , <code>knn_mean_dist_10</code> , <code>coastal_dist</code>
Haversine city distances (6)	AIDE, step 16	great-circle distance to LA / SF / San Diego / Sacramento / Fresno, plus <code>dist_to_nearest_city</code>
Multi-scale KNN spatial encoding (8)	AIDE, steps 17–19	<code>knn_spatial_target_mean_k{5,10,20,50}</code> and <code>knn_spatial_target_std_k{5,10,20,50}</code>

The KNN block is the only genuinely novel modelling idea in the run and the one that produced its two largest single gains. It is computed leakage-safely: train-row values come from an inner `KFold(n_splits=5, shuffle=True, random_state=123)` in which neighbours are drawn only from the inner-training part, while test-row values are computed against the full training coordinate set.

Splitting strategy. The AIDE harness fixes `k_fold_validation: 5`, and every scored node in this tree honours it. The champion goes further and uses **repeated 5-fold KFold with 2 repeats (shuffle=True, seeds 42 and 2024)**, so each of the three algorithms is fitted 10 times and its OOF vector is the average of the two repeats. Fold assignment is identical across the three algorithms within a repeat, which is what makes the OOF blend-weight search legitimate. Unlike the other four episodes in this batch, the leaderboard **was** exercised: the champion's `submission.csv` was submitted and scored, so this report carries both a local and a leaderboard number.

Models & Architecture

Purpose of Architecture. Minimize RMSE on a continuous, strongly spatial target. **Architecture Type** is a **three-member gradient-boosted-tree blend** — LightGBM, XGBoost and CatBoost, combined by a convex weighted average whose weights are grid-searched on the averaged out-of-fold predictions. There is no neural component anywhere in the tree: AIDE drafted a PyTorch MLP at step 6, the import failed, and the repair rewrote it as another LightGBM.

Input Format is a dense float32 matrix — every feature, including the geo-cluster label, is numeric, so no categorical encoding path exists. **Input Dimension** is **40 features per row**, a flat vector with no sequence or image structure.

Architecture Description. The LightGBM member is configured from the inherited `round1_best_lgb_params.json`: `learning_rate 0.011615865989246453`, `num_leaves 121`, `max_depth 10`, `min_child_samples 82`, `subsample 0.6523068845866853`, `colsample_bytree 0.5488360570031919`, `reg_alpha 0.5456725485601477`, `reg_lambda 0.057624872164786026`, `n_estimators 2000`. XGBoost uses `n_estimators 2000`, `learning_rate 0.03`, `max_depth 6`, `subsample 0.8`, `colsample_bytree 0.8`,

`reg_lambda 1.0, tree_method="hist"`. CatBoost uses 2000 iterations, `learning_rate 0.03, depth 8, l2_leaf_reg 3.0, subsample 0.8`. All three early-stop on the fold's validation split with patience 50. The blend layer is a brute-force grid over the 3-simplex in steps of 0.05, scored by OOF RMSE; step 19's analysis records the selected weights as **0.25 LGB / 0.25 XGB / 0.50 CatBoost** — CatBoost, the weakest solo member for most of the run, carries half the final blend.

Model Complexity is best expressed as fits $\times$ trees: the champion performs

3 algorithms $\times$ 2 repeats $\times$ 5 folds = 30 model fits

each with a pre-early-stopping ceiling of 2000 trees (LightGBM at up to 121 leaves, XGBoost at depth 6, CatBoost at depth 8). No dense parameter count is meaningful.

Training procedures

Search trajectory

AIDE's "training" is inseparable from its search, so the staged table below is the trajectory: one row per step, in execution order, with the parent node it mutated. Two steps produced no metric and are marked buggy.

Step	Parent	What AIDE changed	OOF RMSE
0	—	draft: RandomForest, 3 ratio features, raw CSVs	0.59196
1	—	draft: KNN regressor on standardized v2 features	0.66204
2	0	HistGradientBoosting + KMeans geo-cluster	0.56801
3	2	+ ExtraTrees 50/50 blend	0.57373
4	2	HGB 6-config grid search	0.56725
5	—	draft: XGBoost on v2 features	0.56178
6	—	draft: PyTorch MLP	buggy — ModuleNotFoundError
7	—	draft: LightGBM on v2 features	0.56242
8	6	repair of the MLP draft → LightGBM with the inherited params	0.55912
9	8	LGB + XGB weight-searched blend (0.65 / 0.35)	0.55819
10	9	+ CatBoost at full budget	buggy — TimeoutError (1800 s)
11	10	repair: 3-fold, 400 estimators to fit the budget	0.56184
12	9	Ridge stacking meta-learner vs scalar blend	0.55819
13	9	LightGBM regularization grid search	0.55902
14	9	repeated 2 $\times$ 5-fold CV, LGB + XGB	0.55646
15	14	+ CatBoost at full strength, 3-way weight grid	0.55625

Step	Parent	What AIDE changed	OOF RMSE
16	15	+ 6 haversine city-distance features	0.55604
17	16	+ OOF-safe KNN target mean, $k = 10$	0.55254
18	17	+ multi-scale KNN means, $k \in \{5, 10, 20, 50\}$	0.55007
19	18	+ KNN target std at each k (champion)	0.54952

The tree opened with the configured **5 root drafts** (steps 0, 1, 5, 6, 7) and then went deep rather than wide: the champion sits at **depth 8**, on the lineage $6 \rightarrow 8 \rightarrow 9 \rightarrow 14 \rightarrow 15 \rightarrow 16 \rightarrow 17 \rightarrow 18 \rightarrow 19$. Step 9's LGB+XGB blend was the hub of the search, spawning four children (10, 12, 13, 14) of which only the repeated-CV child survived. **12 of the 18 scored steps set a new incumbent** — an unusually productive ratio, driven by the fact that the last six steps improved monotonically once AIDE found the spatial-feature direction.

Total improvement over the search:

$0.59196 - 0.54952 = 0.04244$ (7.1694 % relative reduction)

Three honest weaknesses. First, **the failure modes were environmental, not algorithmic**: step 6 died on `ModuleNotFoundError` because PyTorch is absent from the execution image, and the repair at step 8 abandoned the neural direction entirely and produced another LightGBM — so the "different model family" branch that AIDE's own draft policy asked for never actually executed. Second, **one timeout dominated the compute bill**: step 10 ran the full 1800 s wall-clock limit and produced nothing, which is

$1800 / 2570.2 = 70.03$ % of the run's total execution time

for zero information beyond "this is too slow". AIDE's repair (step 11) then over-corrected by cutting folds and estimators and adding a model at once, scoring 0.56184 — a confounded experiment that AIDE itself diagnosed and re-ran cleanly four steps later at step 15. Third, and least visible: **the champion is the last node of the budget and was still improving**, so the 20-step cap, not convergence, ended the search.

One further piece of context for reading "step 0": this journal is the second leg of the run. The harness was started with `initial_journal` pointing at the 5-node journal of an earlier leg, so AIDE's memory at step 0 already contained five prior attempts on this competition, and its first draft explicitly reasons about "the LightGBM approach already tried in earlier rounds".

Optimization specification

The **Loss Function** is squared error throughout — LightGBM `objective="rmse"`, XGBoost `reg:squarederror`, CatBoost `loss_function="RMSE"` — and model selection is directly on OOF RMSE, the competition metric. The **Optimization Algorithm** is histogram-based **gradient boosting**, not SGD/Adam; on top of it sits a second, non-gradient optimizer, the 0.05-step grid search over blend weights.

The **Learning Rate** is the boosting shrinkage: 0.011615865989246453 for the inherited LightGBM configuration and 0.03 for XGBoost and CatBoost. There is **no Learning Rate Scheduler** (N/A — GBDT convergence is governed by early stopping over boosting rounds, patience 50, not by a schedule) and **no Batch Size** (N/A — full-dataset histogram boosting, not mini-batched).

Training Duration for the whole search was **2570.2 s of executed wall-clock across 20 nodes** (mean 128.5 s, max 1800 s — the timeout). The champion node itself took 68.8 s according to the journal, so the productive part of the run is cheap and the expensive part is the one node that produced nothing. **Training Memory Consumption Limits** are **not recorded** — the AIDE harness caps wall-clock (1800 s per node) but not memory, and no peak-RSS measurement exists.

Transfer Learning is N/A (no pretrained weights; the competition forbids them). Its analogue in an AIDE run is **journal memory**: every step's prompt carries a summary of previously executed nodes and their metrics, which is visibly what drives the plans ("the best result (0.55819) comes from..., while attempts to add a third model hurt"). On this episode a second, unintended transfer channel existed — the pre-engineered CSVs and tuned-LightGBM JSON in the input directory. **Data Augmentation** is N/A (tabular); the analogues are feature engineering (the 14 geo features) and **repeat-seed bagging** (fold seeds 42 and 2024, with each model's seed set to `seed + fold_idx`).

Reproducibility Standards are moderate and unverified. Seeds are fixed everywhere that matters — `np.random.seed(42)` at module scope, KFold seeds 42/2024, per-fold model seeds derived as `seed + fold_idx`, KNN inner-fold seed 123 — but **AIDE has no replay or rebuild gate**: the recorded 0.54952 is the number a single execution printed, and the champion script was never re-run to confirm it reproduces. Any bit-level sensitivity of the LightGBM member at `num_leaves = 121` would therefore go undetected.

Inference procedures

Decision Threshold is N/A — RMSE is a continuous regression metric, so raw real-valued predictions are submitted. **Post-processing is essentially absent**: the test prediction is the mean of the 10 per-fold LightGBM predictions, the 10 XGBoost predictions and the 10 CatBoost predictions, combined with the searched weights, and then written straight out. There is **no clipping to the observed target range, no rounding and no calibration** — the only transformation between blend and file is an id-order realignment against `sample_submission.csv ( set_index("id").loc[sample_sub["id"]] )`. Given that the target of this dataset is bounded, a clip is free accuracy that this run left on the table; AIDE never proposed one in 20 steps.

Inference Duration is **not recorded** as a separate quantity: test prediction happens inside the training loop (each fold model scores the test set immediately after fitting), so it is folded into the 68.8 s of the champion node. **Inference Memory Consumption Limits** are **not recorded**; scoring 24,759 rows through 30 tree models is negligible on CPU.

Evaluation & Benchmarking

Task for Performance Evaluation. Predict `MedHouseVal` for each of the 24,759 test census blocks and submit a two-column CSV in `sample_submission.csv` order. **Performance Metrics.** RMSE, minimized. AIDE's local OOF trajectory runs $0.59196 \rightarrow 0.56178 \rightarrow 0.55819 \rightarrow 0.55625 \rightarrow 0.54952$ across the draft, the first native GBDT, the first blend, the first three-way blend and the final spatial-feature champion. Local-to-leaderboard degradation is small and in the expected direction:

```
0.55343 (private) - 0.54952 (local OOF) = 0.00391
0.55863 (public)  - 0.55343 (private)   = 0.00520
```

so the 5-fold OOF estimate was only mildly optimistic and the public/private split is well-behaved. The submission placed **218/690, the 68.6th percentile**.

Performance Benchmarking. All three lanes submitted to the same private leaderboard:

Lane	Approach	Private RMSE	Note
AIDE	20-step tree search, repeated-CV GBDT blend + KNN spatial encoding	0.55343	lb_winner
NVIDIA	reproduce-agent, public-kernel replication	0.5555	behind by 0.00207
My agent	from-scratch GBDT pool + tree search	0.55987	behind by 0.00644

```
0.5555 - 0.55343 = 0.00207
0.55987 - 0.55343 = 0.00644
```

AIDE wins this episode on the leaderboard, and the mechanism is identifiable: steps 17–19 (KNN spatial target mean, then multi-scale k, then local dispersion) moved the local metric $0.55604 \rightarrow 0.54952$, which is more than the entire preceding blending programme achieved. That is the tree search working as designed — a cheap, local, comp-specific idea found by mutation and confirmed by execution.

Two caveats must travel with the result. First, `facts_aide.json` records `local_winner = nvidia` while `lb_winner = aide`; the other lanes' local scores are outside this report's source scope, so the disagreement is reported, not resolved. Second, and more material: as documented in the Data section, this is the one episode where AIDE's input directory contained pre-engineered features and a tuned-LightGBM parameter file, both of which the champion consumes directly. AIDE's margin here therefore reflects an inherited feature baseline plus its own geo additions — not a clean from-raw-data win.